Um operador bitwise em Java (*bitwise operator in Java*) permite que você manipule bits individualmente, da maneira como preferir. Ele pode ser útil para otimizar códigos, implementar máscaras de bits e flags, compactar dados, e implementar algorítmos e estruturas de dados. Os sete operadores bitwise existentes têm estruturas semelhantes e seguem regras fixas, fáceis de aprender. Nosso tutorial ensina a você.

O que são e para que servem operadores bitwise em Java?

Operadores em Java são elementos de fundamental importância desta linguagem de programação web. Além de permitirem diversos tipos de ações e realizarem diferentes funções em códigos, operadores em Java possibilitam a manipulação das menores unidades de informação existentes: os bits. Bits (*binary digits*) adotam um sistema de numeração próprio, baseado nos valores 0 e 1. Por isso, para verificar valores binários em cada bit, programadores devem apelar aos operadores bitwise. Estes podem ser aplicados a **tipos de dados numéricos**, como a *byte*, *char*, *int*, *short* e *long*.

Embora você não seja obrigado a fazer uso do Java bitwise, aprender a utilizá-lo pode ser de extrema utilidade. *Bitwise operators in Java* **economizam espaço ao converter dados** e permitem a aplicação do **operador XOR**, assim como possibilitam a **alteração de um único bit**. Ainda, eles funcionam como base para todas as operações técnicas envolvendo circuitos mais complexos. Por esses e outros motivos, recomendamos que você conheça o funcionamento e a aplicação dos sete operadores Java bitwise.

Web hosting

Get your site online fast without breaking the bank

- Fast page loads while cutting costs by 60%
- Keep your site open 24/7 with 99.99% uptime
- Never run out of space with unlimited storage

See plans

Quais são os tipos de operadores bitwise em Java?

Existem sete tipos de operadores Java bitwise. Para que possamos explicá-los melhor, pediremos ajuda a dois operandos fictícios, que chamamos aqui de “a” e “b”.

- `~` **(NOT Bit-a-Bit)**: Este operador realiza uma operação *NOT bitwise*, invertendo cada um dos bits. Assim, cada “0” será transformado em “1” e vice-versa.
- `&` **(AND Bit-a-Bit)**: Este operador realiza uma operação *AND bitwise* entre cada par de bits, retornando “1” se ambos os operandos também forem “1”. Caso contrário, “0”será retornado.

- `|` **(OR Bit-a-Bit)**: Este operador realiza uma operação *OR bitwise* entre cada par de bits, produzindo “1” se pelo menos um dos operandos também tiver esse valor.
- `^` **(XOR; OR exclusivo Bit-a-Bit)**: Este operador realiza uma operação *XOR (OR exclusivo) biwise*, retornando “0” se ambos os operandos tiverem o mesmo valor. Caso contrário, ele retornará “1”.
- `<<` **(Deslocamento para a esquerda)**: Este operador desloca o operando “a” por “b” posições para a esquerda. Se a operação gerar espaços vazios, estes serão preenchidos com “0”.
- `>>` **(Deslocamento para a direita com sinal; aritmético)**: Este operador desloca os bits de “a” por “b” posições para a direita. Se o bit de sinal já tiver sido definido, este permanecerá definido; números negativos permanecem negativos.
- `>>>` **(Deslocamento para a direita sem sinal; lógico)**: Este operador desloca os bits de “a” por “b” posições para a direita; espaços vazios são sempre preenchidos com “0”, ignorando o bit de sinal.

Java bitwise NOT Bit-a-Bit

O operador Java bitwise *NOT Bit-a-Bit* é representado pelo til (`~`). Ele **nega todos os bits**, transformando cada zero (0) em um (1), e vice-versa. Considere, por exemplo, o número 20: em formato binário, ele é representado como *10100*. Assim, ao aplicarmos esse *bitwise operator in Java*, invertemos um dos bits (*10100* se torna *01011*) — esse é o valor da expressão `~20` . Agora, ao convertermos o resultado binário para decimal, obtemos *-21*.

Se você quer testar a operação bitwise *NOT Bit-a-Bit* em um código, insira este comando em Java, `System.out.println` , no seu terminal, para obter a saída correspondente:

```
public class Main {
  public static void main(String[] args) {
  int Valor1 = 20;
  System.out.println(~Valor1);
}
}
```

java

Se você tiver digitado tudo corretamente, deverá obter o resultado *-21*.

Java bitwise AND Bit-a-Bit

O operador Java bitwise *AND Bit-a-Bit* (`&`) **compara dois números, bit a bit, em formato binário**: o primeiro bit do primeiro número é comparado ao primeiro bit do segundo número; o segundo com o segundo bit e assim por diante. Se ambos os bits forem *1*, o valor *1* será retornado. Contudo, se ambos os bits ou apenas um deles for *0*, o valor *0* será retornado. A

título de exemplo, usamos os números decimais *18* e *25*. Na forma binária, 18 é representado por *10010* e 25 por *11001*. Com o operador bitwise AND, desejamos comparar esses dois números, obtendo um terceiro número como resultado — para uma melhor compreensão, posicionamos os dois números binários um abaixo do outro:

18 = 10010 25 = 11001

Observe que os dois primeiros bits são 1 em ambos os números. Assim, nosso número de saída também começará com 1. Embora o segundo bit de 25 também seja 1, o segundo bit de 18 é 0, o que fará com que o resultado dessa operação seja 0. Ao compararmos todos os bits, um a um, finalmente obteremos o número binário *10000* que, convertido em número decimal, torna-se 16.

No nosso código, executamos essa operação bitwise em Java da seguinte forma:

```
public class Main {
  public static void main(String[] args) {
  System.out.println(18&25);
}
}
```

java

Ao colar esse código no seu terminal, você obterá o resultado *16*.

Java bitwise OR Bit-a-Bit

O operador Java bitwise *OR Bit-a-Bit* (`|`) também compara pares de bits. Ao utilizá-lo, bastará que apenas um dos dois operandos tenha o valor 1 para que o resultado também passe a ser 1. Tomemos novamente os números 18 e 25 como exemplo:

18 = 10010 25 = 11001

Como todos os bits, exceto o terceiro, contêm pelo menos um valor *1*, o valor gerado pela operação é *11011*, equivalente ao número decimal *27*.

Assim seria a aplicação, em um código, desse *bitwise operator in Java*:

```
public class Main {
  public static void main(String[] args) {
  System.out.println(18|25);
}
}
```

java

Java bitwise XOR

O operador Java bitwise *XOR*, ou OR exclusivo (`^`), é semelhante ao operador bitwise *OR*. No entanto, enquanto *OR* exige que um ou ambos os operandos sejam 1, *XOR* só retorna 1 se **exatamente um dos dois valores** também for 1. O exemplo abaixo esclarece o conceito:

18 = 10010 25 = 11001

Observe que os dois primeiros bits têm o valor *1*, que é transformado em 0 por este operador Java bitwise. Por outro lado, o segundo bit de 18 é *0* enquanto o de 25 é *1*. Assim, o valor resultante também será *1*. Ao executarmos a operação em todos os bits, obtemos esta terceira combinação: *01011*, equivalente ao número decimal *11*.

O código dessa operação bitwise XOR é o seguinte:

```
public class Main {
  public static void main(String[] args) {
  System.out.println(18^25);
}
}
```

java

Java bitwise de deslocamento para a esquerda

Uma operação Java bitwise de deslocamento para a esquerda (`<<`) desloca os bits de valor “a” para a esquerda, pela distância “b”. Nesse tipo de operação, **espaços em branco do resultado são preenchidos com 0**. Exemplo clássico é o valor int que ocupa 32 bits de uma memória. Nosso exemplo tomará o número decimal 20 (*10010*) e o deslocará pelo valor “b” 2, para obter o valor “c” *1001000*. Observe que, nesse caso, dois zeros são adicionados ao final da sequência: *1001000* (correspondente ao número decimal 72).

Para deslocar bits para a esquerda com uma operação Java bitwise, baseie-se no exemplo de código abaixo:

```
public class Main {
  public static void main(String[] args) {
  int a = 20;
  int b = 2;
  int c = (a << b);
  System.out.println(c);
}
}
```

java

Java bitwise de deslocamento para a direita com sinal

Uma operação Java bitwise de deslocamento para a direita com sinal (`>>`) funciona da maneira exatamente oposta à operação de deslocamento para a esquerda. Como o nome já diz, o deslocamento à direita move, para a direita, os bits do valor “a” por um valor “b” , obtendo o valor “c”. Nesse tipo de operação, **os últimos bits são eliminados**. No nosso exemplo, deslocamos o valor 20 (*10010*) duas posições para a direita, obtendo o resultado *100* (número decimal 4).

O código abaixo detalha a operação:

```
public class Main {
  public static void main(String[] args) {
  System.out.println(20 >> 2);
}
}
```

java

Observe que a operação Java bitwise de deslocamento à direita preenche espaços em branco com *0*, mas somente quando “a” é um número positivo. Caso ele seja negativo, os espaços em branco serão preenchidos com 1.

Java bitwise de deslocamento para a direita sem sinal

O operador Java bitwise de deslocamento para a direita sem sinal (`>>>`) funciona basicamente da mesma maneira que o outro operador de deslocamento para a direita. A diferença está no fato de que o operador para a direita sem sinal **sempre preenche com *0*** os espaços em branco que surgem do lado esquerdo do valor deslocado para a direita. Dessa forma, essa operação bitwise sempre resultará em valores positivos, mesmo que os valores iniciais sejam negativos.